Software Quality

Quality here means code that stays consistent, simple, and low-complexity — enforced by tooling in CI, not by memory. Tests and coverage are covered separately in Testing; this handbook is about **style, conventions, and static analysis**.

Back-end — Checkstyle

The API is checked against a custom **Checkstyle** config (`checkstyle.xml`) covering naming (`TypeName`, `MethodName`, `MemberName`, `ConstantName`), imports (no star imports, no unused/redundant imports), whitespace, braces (`NeedBraces`), modifier order, and indentation. Lines are capped at **120** characters.

It also caps complexity, so methods stay small and readable:

Rule	Limit
Method length	60 lines
Parameters per method	7
Cyclomatic complexity	10
NPath complexity	200
Anonymous inner class length	20 lines

A companion `checkstyle-suppressions.xml` relaxes a few rules where they add noise: controllers skip Javadoc-on-methods (Spring annotations are self-explanatory), entities may use magic numbers, and generated MapStruct mappers, controllers, and test sources are exempt from line-length and indentation checks.

Checkstyle runs in CI (`mvn … checkstyle:checkstyle`) and the HTML report is uploaded as a build artifact.

Front-ends — ESLint & Prettier

Both Angular apps lint with **ESLint 9** using `@angular-eslint` and `@typescript-eslint`, and format with **Prettier**:

```
npm run lint      # ng lint (ESLint)
npm run format    # prettier --write .
```

`npm run lint` runs in CI for both the app and the dashboard. An `.editorconfig` fixes the shared basics — UTF-8, **2-space** indentation, a final newline, trimmed trailing whitespace, and **single quotes** in TypeScript. The dashboard adds a few extra ESLint plugins (`import`, `jsdoc`, `prefer-arrow`). The mobile app has **husky** available for Git hooks, though linting currently runs manually and in CI rather than on commit.

Static analysis

Two layers of static analysis run on top of linting.

DeepSource runs continuously across all three repositories (`.deepsource.toml`):

- **API** — the Java analyzer (runtime 21), with `test_patterns` narrowed to `**/*Test.java` so the shared fixtures (`TestApp`, `TestEntity`, `TestGetDTO`, `TestPostDTO`, `TestHelper`) aren't flagged as empty test files.
- **Mobile app** — the JavaScript analyzer with the Angular plugin (Node, Jasmine, and Cypress environments); a couple of rules are skipped where Angular's patterns trip them (empty module classes `JS-0327`, required-input aliasing `JS-0585`).
- **Dashboard** — the JavaScript analyzer with the Angular plugin (Node, Jasmine).

JetBrains Qodana adds a second pass (the `qodana.starter` profile), but only on the **API** (`qodana_code_quality.yml`, JVM community linter). On the mobile app a Qodana workflow exists but is **disabled** (`.yml.disabled`), and the dashboard has none. Qodana's quality-gate thresholds in `qodana.yaml` are left commented out, so it reports findings without failing the build on a count *(to confirm)*.

Where quality is enforced

Repository	Style	Analysis
`candex-api`	Checkstyle	DeepSource + Qodana
`candex-app`	ESLint + Prettier	DeepSource
`candex-dashboard`	ESLint + Prettier	DeepSource

The API also has the JaCoCo 100% coverage gate — see Testing. Linting and Checkstyle run in CI on each push and pull request, DeepSource analyses every change continuously, and a change is not done until everything is green. For the full CI picture and the test/coverage gates, see Testing.

Where to go next

- Tests and coverage → Testing
- The code these rules apply to → Back-end • Front-end